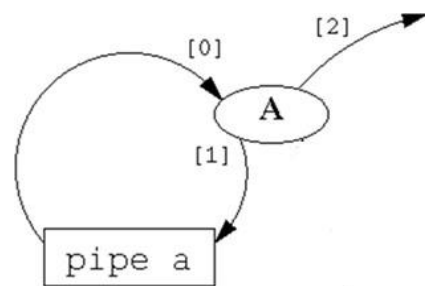


Operating Systems

CS-2006

Question 1

Suppose the following code sections are executed without any error. Draw the diagrams showing the relationship between the file descriptors, pipes and processes (first one is given).

Code section	Diagram after the execution of the code section (A) is the name of parent process.
<pre>int fda[2], fdb[2]; pipe(fda); dup2(fda[0], STDIN_FILENO); dup2(fda[1], STDOUT_FILENO); close(fda[0]); close(fda[1]);</pre>	 The diagram shows a process represented by an oval labeled 'A'. Two arrows originate from process A: one labeled '[0]' that curves down and left to point at a rectangular box labeled 'pipe a', and another labeled '[1]' that curves down and left to also point at the 'pipe a' box. A third arrow labeled '[2]' originates from process A and points up and to the right, away from the pipe.
<pre>pipe(fdb);</pre>	
<pre>fork();</pre>	

Note: In the diagram, [2] is Error fd, and the arrow shows that it is in its real state.

// dup2(int fd1, int fd2); closes fd2 and copies fd1 to fd2

// dup(int fd); copies fd to smallest available file descriptor i.e., firstly 1 then 2 and so on...

Question 2

Assume that we have the following piece of code:

```
01|  int main() {
02|  printf("Starting main\n");
03|  int file_fd = open("test.txt", O_WRONLY | O_TRUNC | O_CREAT, 0666);
04|  dup2(file_fd, STDOUT_FILENO);
05|  pid_t child_pid = fork();
06|  if (child_pid != 0) {
07|      wait(NULL);
08|      printf("In parent\n");
09|  }
10|  else {
11|      printf("In child\n");
12|  }
13|  printf("Ending main: %d\n", child_pid);
14|  } // end of main()
```

What is the output of this program? You can assume that no syscalls fail and that *the child's PID is 1234, and parent's PID is 4841*. Fill in the following table with your prediction of the output:

Standard out	test.txt

Next, assume we have altered the code as follows:

```
01|  int main() {
02|  printf("Starting main\n");
03|  int file_fd = open("test.txt", O_WRONLY | O_TRUNC | O_CREAT, 0666);
04|  int new_fd = dup(STDOUT_FILENO);
05|  dup2(file_fd, STDOUT_FILENO);
06|  pid_t child_pid = fork();
07|  if (child_pid != 0) {
08|      wait(NULL);
09|      printf("In parent\n");
10|  }
11|  else {
12|      dup2(new_fd, STDOUT_FILENO);
13|      printf("In parent\n");
14|  } // end of else block
15|  printf("Ending main: %d\n", child_pid);
16|  } // end of main()
```

What is the output this time? Fill in the following table with your prediction of the output:

Standard out	test.txt

Question 3

Consider the following program, executing under **normal conditions**:

```
int main()
{
    int fd[2];
    pipe(fd);
    char buf[5];
    printf("FORK\n");
    if (fork()) {
        int rc = read(fd[0], buf, 4);
        switch (rc) {
            case 4:
                buf[4] = 0;
                printf("%s\n", buf);
                break;
            case 0:
                printf("READ-EOF\n");
                break;
            case -1:
                printf("READ-ERROR\n");
                break;
        }
    } // end of if block
    else {
        dup2(fd[1], STDOUT_FILENO);
        int rc = execv("/bin/echo", (char* []) {"echo", "ECHO", NULL});
        switch (rc) {
            case 0:
                printf("EXEC-SUCCESS\n");
                break;
            case -1:
                printf("EXEC-ERROR\n");
                break;
        }
    } // end of else block
    printf("DONE\n");
}
```

Which of the following options can be the possible output of the above code? **Select all possible outputs.**

☐	FORK ECHO EXEC-SUCCESS DONE DONE	☐	FORK FORK ECHO DONE DONE
☐	FORK READ-EOF DONE	☐	FORK ECHO DONE
☐	FORK FORK ECHO EXEC-SUCCESS DONE DONE	☐	FORK READ-ERROR EXEC-ERROR DONE DONE

Question 4

Consider the following C program, executing on Linux.

```

01| pid_t save_cpid;
02| int main()
03| {
04|     pid_t cpid = fork();
05|     if (cpid < 0) {
06|         perror("fork"), exit(1);
07|     }
08|     save_cpid = cpid;
09|     if (cpid == 0) {
10|         printf("Child is %d and parent is %d\n", getpid(), getppid());
11|         save_cpid = save_cpid + 25;
12|         sleep(10); // sleep for 10s
13|     }
14|     else {
15|         printf("Parent is %d and child is %d\n", getpid(), cpid);
16|         save_cpid = save_cpid - 25;
17|     }
18|
19|     printf("%d sees save_cpid = %d\n", getpid(), save_cpid);
20|     if (cpid == 0) {
21|         exit(42);
22|     }
23|     int child_status;
24|     pid_t wpid = wait(&child_status);
25|
26|     if (wpid == -1)
27|         printf("Could not wait for child yet.\n");
28|     else if (WIFEXITED(child_status))
29|         printf("Child %d terminated with exit status %d\n",
30|             wpid, WEXITSTATUS(child_status)); // part of else if
31|     else
32|         printf("Child %d terminated abnormally\n", wpid);
33|     return 0;
34| }

```

When the above program was executed, lines 10 and 15 produced the following output:

Parent is 8386 and child is 8387

Child is 8387 and parent is 8386

(though not necessarily in this order).

(A): What output would have been produced by line 19?

(B): Which, if any, of lines 27, 29 or 32 will be executed, and what would be output by the line(s)?

If the answer cannot be uniquely determined in a concurrent environment, say why.

Question 5

Implementation of a UNIX Command:

The `/usr/bin/time` command can be used to determine how long a **Unix command** takes to execute as well as additional statistics about its execution. The command takes the name and command line arguments of an arbitrary program as input, and passes those command line arguments to the program it executes.

For instance, below are examples that show **how to time the execution of 'sleep 1.5' and 'echo Hello World'**.

```
$ /usr/bin/time sleep 1.5
0.00user 0.00system 0:01.50elapsed
```

```
$ /usr/bin/time echo Hello World
Hello World
0.00user 0.00system 0:00.00elapsed
```

In the above example, the first command took 1.5 seconds to complete, as it is sleep 1.5, and in the second example it executed without any delay.

In this Question, you should implement a simplified version of time which only prints the real (wall clock) time elapsed during the execution of any command. **The output of your command shall be:**

```
$ ./time sleep 1.5
Took 1 sec 502470 usec.
```

```
$ ./time echo Hello World
Hello World
Took 0 sec 960 usec.
```

Complete the implementation of time.c Program, carefully analyze the execution command of executable file of time.c, and implement your logic accordingly.

```
01: int main(int ac, char* av[]) {
02:
03:     struct timeval start, end, diff;
04:     gettimeofday(&start, NULL);
05:     /* Complete this part, for simplicity no need to show error checking */
06:     if (_____) {
07:         _____;
08:     }
```

```

09: _____;
10: gettimeofday(&end, NULL);
11: timersub(&end, &start, &diff);
12: printf("Took %ld sec %ld usec.\n", diff.tv_sec, diff.tv_usec);
13: return 0;
14: } // end of main()

```

Question 6

Implementation of pipe Command:

You are asked to write a program (say prog.c) that forks and runs a sub-shell as a child process, and simply counts the number of output characters that the shell **printed on the standard output**. When the sub-shell terminates, the parent, simply reports the number of output characters produced by the sub-shell.

Some part of the prog.c is already implemented for you, you can ignore most of the error checking to make your solution clear, **but you need to close all necessary file or pipes and check what read-write etc., returns. Also read/write one char at a time to make counting job easy!**

```

/* Implementation of prog.c */
/* There may be some extra lines given before while loop, you may leave them */
/* Only implement the required functionality on each blank block of lines */
/* Pipes and file descriptors must be set acc. To above req. statement */

01: int main(int ac, char* av[]) {
02:     int pfd[2];
03:     int childpid, numread, numwritem, count=0;
04:     char buf;
05:     /* Setup the pipe and child process below */
06:     _____;
07:     _____;
08:     _____;
09:     /* start child execution below */
10:     if (_____) {
11:         /* child sets up its pipe below */
12:         _____;
13:         _____;
14:         _____;
15:         _____;
16:         execl("/bin/bash", "shell", NULL);
17:         perror("cannot start shell");
18:         return 1;
19:     } // end of if statement
20:
21:     /* parent sets up the pipe below */
22:     _____;
23:     _____;
24:     _____;
25:     _____;
26:     /* parent reads from pipe as long as there is data */

```

```

27:    /* counts the characters and puts them on standard output */
28:    while (_____) {
29:        numread = read(_____, &buf, ____);
30:        if (_____) break;
30:        _____;
31:        numwrite = write(_____, _____, ____);
32:        if (_____) break;
33:    } // end of while
34:    fprintf(stderr, Shell printed %d characters.\n, ____);
35:    /* process terminates (1 important thing to do before this) */
36:    _____;
37: } // end of main()

```

Question 7

Exec and Unnamed Pipes:

As we know that there are two types of pipes, Unnamed and Named. Unnamed Pipes are used for communication between related processes. Now suppose we have two C programs given below:

Prog.c	testExec.c
<pre> int main() { int pfd[2]; pipe(pfd); pid_t id = fork(); if (id == 0) execl("/home/aneeq/Desktop/OS/testExec", "./testExec", NULL); wait(NULL); // reap the child // close the write end close(pfd[1]); char buf[17]; read(pfd[0], buf, 17); printf("%s", buf); } </pre>	<pre> int main() { close(3); char buf[] = "Hello from Child"; write(4, buf, sizeof(buf)); } </pre>

In the above example, we are communicating between two different programs using unnamed pipes, because they both are being executed by related processes (parent-child). But, if you notice we don't have the access of pipe variable **int pfd[2]** in testExec.c, and we have to manually put the descriptor. Examining this scenario answer the following Questions:

(A): Will both programs communicate effectively, i.e., Will **“Hello from Child”** be printed in Prog.c?

(B): How can we access the variable **int pfd[2]** in testExec.c file? Make all necessary changes in the code for this implementation. After successful implementation we should be able to do

```
close(pfd[0]);
write(pfd[1], buf, sizeof(buf));
```

in testExec.c, rather than to use 3 & 4.

(C): Modify the program as such that we take **No. of pipes** input from user. Set up all those pipes in Prog.c, and access all those pipes in testExec.c, also basic implementation of above program must stay same, i.e., closing write ends of all pipes in Prog.c and read ends of all pipes in testExec.c and finally read and write from all pipes as same as the above program.

Question 8

Shells use the UNIX system call `pipe( )` to connect a program's standard output to another program's standard input. Following is an incomplete example program that shows how redirection works. It implements a program "spipe.c", which starts two child processes whose standard input/outputs connected through a pipe.

In other words, `./spipe program1 program2` is equivalent to running `program1 | program2` in a shell that supports pipes:

```
$ ./spipe ls sort
// displays the list in sorted way

$ ls | sort
// displays the list in sorted way
```

For simplicity, spipe does not support passing on any arguments to the commands it launches. For presentation purposes, error handling was omitted as well. Complete the program given:

```
/* Implementation of spipe.c */
/* Following function is already implemented for you: */
/* int launch_child(char* prgname, char* prgargv[], int stdout, int stdin,
                    int fdtoclose ) */

/* Explanation of the above stated function: */
/* Launches a child process, redirect its standard out to stdout, standard in to
   stdin, Also close 'fdtoclose' in child. Close any redirected file descriptors
   In parent before returning. Returns pid of child process */

01: int main(int ac, char* av[]) {
02:
03:     char* firstprogargv[] = { av[1], NULL };
04:     char* secondprogargv[] = { av[2], NULL };
05:     /* Setup the pipe below */
06:     _____;
07:     _____;
08:
09:     /* launch first child */
10:     pid_t leftchild = launch_child(av[1], firstprogargv, _____, _____, _____);
11:     /* reap the first child below use waitpid */
12:     _____;
13:     /* launch second child */
```



```

14:     pid_t rightchild = launch_child(av[2], secondprogargv, ____, ____, ____);
15:     /* reap the first second below use waitpid */
16:     _____;
17: }

```

Users observed the following behavior of “spipe”. For many combinations of programs, “spipe” appears to work as intended. For some (say prog2.c, implementation hidden) “it gets stuck”, then the ps command shows:

```

$ ps f
  PID TTY          STAT       TIME COMMAND
 24889 pts/9    Ss          0:00 -bash
 31462 pts/9    S           0:00 \_  ./spipe ./prog2 wc
 31463 pts/9    S           0:00 |      \_  ./prog2
 31490 pts/9    R+          0:00 \_  ps f

```

Based on this information, answer the following Questions:

- (A) Does this apparent bug of “spipe” cause excessive CPU usage on the machine on which user runs it? Justify your answer!
- (B) Why does ‘spipe’ not finish? Be precise!
- (C) Suggest a way to fix spipe!

Question 9

Suppose that the file “tmpdata.txt” contains “abcdefghijk”. If the following code is executed correctly without generating any errors, then:

```

01: int main(int ac, char* av[]) {
02:
03:     int fd;
04:     char buf[6] = “12345”;
05:     fd = open(“tmpdata.txt”, O_RDONLY);
06:     fork();
07:     read(fd, buf, 2);
08:     read(fd, buf + 2, 2);
09:     printf(“%d: %s\n”, getpid(), buf);
10: } // end of main()

```

- (A) Explain if the following two outputs are possible or not? Why/why not?
Suppose parent’s pid is 7 while child’s pid is 8.

7:abcd5 8:abef5	
7:abef5 8:cdgh5	

- (B) What could be the outputs if the lines 5 & 6 are exchanged? Write at least 3 possible outputs.

Question 10

Suppose the following C code is executed; assume appropriate wait() calls occur in the missing part of the code. The executables progA, progB, progC and progD are installed in directories in the system path, and each simply prints a message indicating the pid of the process executing it and then terminates.

```
/* implementation of fork02.c */
01: int main(int ac, char* av[]) {
02:
03:     char* argv1[] = {"progA", NULL};
04:     char* argv2[] = {"progB", NULL};
05:     char* env[] = {NULL};
06:
07:     printf("%d is entering fork02.\n", getpid());
08:     int index = 0;
09:     pid_t pid = fork();
10:     pid = fork();
11:     if ( pid == 0 && index == 0 ) {
12:         index++;
13:         pid = fork();
14:         if ( pid == 0 ) {
15:             execve("progA", NULL, NULL);
16:         } // end of nested if
17:
18:         if ( pid != 0 && index = 0 ) {
19:             pid = fork();
20:             execve("progB", NULL, NULL);
21:         } // end of nested if
22:     } // end of if
23:     /* Assume Necessary wait calls executed here */
24:     printf("%d is exiting from fork02.\n", getpid());
25:     return 0;
26: } // end of main()
```

- (A) Which program is the original process executing when that original process terminates?
- (B) How many processes terminate in each of the relevant programs?

Program	# of processes that terminate there
fork02	
progA	
progB	

Question 11

Consider the following example programs. List all possible outputs.

<pre>// included in all programs #include <unistd.h> #include <sys/wait.h> // W(A) means write(1, "A", sizeof "A") #define W(x) write(1, #x, sizeof #x)</pre>	<u>Possible Outputs:</u>
<pre>01: int main() { 02: 03: W(A); 04: fork(); 05: W(B); 06: fork(); 07: W(C); 08: } // end of main()</pre>	
<pre>01: int main() { 02: 03: W(A); 04: int child = fork(); 05: W(B); 06: if(child) 07: wait(NULL); 08: W(C); 09: } // end of main()</pre>	
<pre>01: int main() { 02: 03: W(A); 04: if(!fork() && !fork()) 05: W(B); 06: else { 07: W(A); 08: exit(0); 09: } // end of else 08: W(C); 09: wait(NULL); 10: } // end of main()</pre>	

Question 12

What output do the following 2 programs produce & why?

<pre>#include <pthread.h> #include <stdio.h> int counter; static void * thread_func(void * _tn) { int i; for (i = 0; i < 100000; i++) counter++; return NULL; } // end of function int main() { int i, N = 5; pthread_t t[N]; for (i = 0; i < N; i++) pthread_create(&t[i], NULL, thread_func, NULL); for (i = 0; i < N; i++) pthread_join(t[i], NULL); printf("%d\n", counter); return 0; }</pre>	<pre>#include <pthread.h> #include <stdio.h> int counter; static void * thread_func(void * _tn) { int i; for (i = 0; i < 100000; i++) counter++; return NULL; } // end of function int main() { int i, N = 5; pthread_t t[N]; for (i = 0; i < N; i++) { pthread_create(&t[i], NULL, thread_func, NULL); pthread_join(t[i], NULL); } // end of for loop printf("%d\n", counter); return 0; }</pre>
<u>Output:</u>	<u>Output:</u>

Question 13

Consider the following 2 programs and predict their output.

<pre># define size 6 int main() { int pfd[2]; pipe(pfd); if(!fork()) { close(pfd[0]); } }</pre>	<pre># define size 6 int main() { int pfd[2]; pipe(pfd); if(!fork()) { close(pfd[0]); } }</pre>
--	--

<pre> char buf[size] = "Ooops"; write(pfd[1], buf, sizeof(buf)); write(pfd[1], buf, 3); } // end of if else{ close(pfd[1]); char buf[size * 2]; read(pfd[0], buf, size + 1); printf("%s\n", buf); char buf2[size]; read(pfd[0], buf2, 3); printf("%s", buf2); } // end of else } // end of main() </pre>	<pre> char buf[size] = "Ooops"; write(pfd[1], buf, sizeof(buf)); write(pfd[1], buf, 3); } // end of if else{ close(pfd[1]); char buf[size * 2]; read(pfd[0], buf, size + 5); printf("%s", buf); } // end of else } // end of main() </pre>
<u>Output:</u>	<u>Output:</u>

Question 14

Consider the following code snippets, identify No. of Zombie & Orphan processes, when control of parent process reaches the sleep().

<pre> int main() { if(!fork()) { printf("Hello from child 1"); wait(NULL); exit(0); } else { if(!fork()) { wait(NULL); exit(0); } sleep(10); } } </pre>	<pre> int main() { if(!fork()) { if(!fork()) { exit(0); // exit-1 } exit(0); // exit-2 } else { printf("Hello from parent"); } sleep(10); } </pre>
No. of Zombies:	No. of Zombies:
No. of Orphans:	No. of Orphans:

Question 15

Dividing the Work:

Consider the following incomplete program (**Max.c**) which finds the maximum value in an array, but NOT with a single process. It takes multiples processes and divides the array between them, and each finds the maximum in their array and writes to pipe.

Complete the code given below:

```
/* Implementation of Max.c */
01: int max(int *arr, int start, int end) {
02:     int maxVal = arr[start];
03:     for (int i = _____; i < end; i++)
04:         if(arr[i] > maxVal) maxVal = arr[i];
05:
06:     return maxVal;
07: } // end of function
08:
09: int main() {
10:     srand(time(0));
11:     int size, Np; // No. of processes
12:     printf("Enter size of array: ");
13:     scanf("%d", &size);
14:
15:     int *arr = (int *)malloc(sizeof(int) * size);
16:     printf("Array: ");
17:     for (int i = 0; i < size; i++) {
18:         arr[i] = rand() % 100 + 1;
19:         printf("%d ", arr[i]);
20:     } // end of for loop
21:
22:     printf("\nEnter No. of processes: ");
23:     scanf("%d", &Np);
24:
25:     int sizePerProcess = _____, start = 0;
26:
27:     /* set up the pipe below */
28:     _____;
29:     _____;
30:
31:     for (int i = 0; i < Np; i++) {
32:         if (_____){ // child process should execute
33:             _____;
34:             int m = max(arr, start, _____);
35:             write(_____, _____, _____);
36:             exit(0);
37:         } // end of if
38:         _____;
39:     } // end of for loop
40:     _____; // close pipe for parent
```

```

41:    /* reap all the zombie processes below */
42:    _____;
43:    _____;
44:
45:    int m, p=1;
46:    read(_____, &m, _____); // read first value written to pipe
47:
48:    for (int i = 0; i < _____; i++) {
49:        int newM;
50:        read(_____, _____, _____);
51:        if (newM > m) {
52:            m = newM;
53:            p = _____;
54:        }
55:    } // end of for loop
56:    printf("\nMax of array: %d found by Process No. %d", m, p);
57: } // end of main()

```

Now as you can analyze, how we divided the work between the processes. Answer the following Questions by analyzing the completed code:

- (A) Is this calculation of max, running in Parallel or Serial?
- (B) If answer to above is serial, then convert to Parallel and vice versa.
- (C) Convert the above code, by using threads instead of processes.

Question 16

Suppose the following C program is executed, and the executable alpha and beta exist in the current directory

```

void rptNewChild(pid_t pid, char* src)
{
    if (pid != 0) {
        printf("%d forked %d at %s\n", getpid(), pid, src);
    }
}

void chill(pid_t pid)
{
    pid_t cpid;
    printf("%d is chilling\n", pid);
    while ((cpid = wait(NULL)) > 0) {
        printf("%d waited for %d\n", pid, cpid);
    }
}

int main() {
    char* av[] = { NULL };
    char* env[] = { NULL };

    printf("%d is the original process.\n", getpid());

```

```

pid_t pid = fork(); // F1 fork
rptNewChild(pid, "F1");

if (pid != 0) {
    pid = fork();
    rptNewChild(pid, "F2");
    if (pid == 0) {
        pid = fork();
        rptNewChild(pid, "F3");
        chill(getpid());
        execve("./alpha", av, env);
    }
}
else {
    chill(getpid());
    execve("./beta", av, env);
    pid = fork();
    rptNewChild(pid, "F4");
}
chill(getpid());

return 0;
}

```

Alpha.c	Beta.c
<pre> int main(int argc, char** argv) { printf("%d running alpha\n", getpid()); sleep(5); return 0; } </pre>	<pre> int main(int argc, char** argv) { printf("%d running beta\n", getpid()); sleep(5); return 0; } </pre>

When the program was executed, the various calls to `rptNewChild( )` produced the following output (and possibly more that is not shown here):

20317 is the original process.

20317 forked 20318 at F1

20317 forked 20319 at F2

20319 forked 20320 at F3

Based on this information answer the following Questions:

Questions	Answers
State the PIDs of the processes, if any, that will eventually execute the program alpha.	
State the PIDs of the processes, if any, that will eventually execute the program beta.	

Will a child be forked at the <code>fork()</code> call labelled F4? If so, state the PID of parent that will fork that child, and assume the child's PID is 20321. If not, explain why no child will be forked there.	
State the PIDs of the processes, if any, that will eventually call <code>chill()</code>.	
Determine which processes will eventually produce output from <code>printf()</code> call inside while loop in function <code>chill()</code>. For each of those processes, write output that would be produced.	
Is it possible to determine which process will terminate last? If so, state the PID of that process & explain why it cannot terminate until all the others have done so. If not, list two different orders in which the processes could terminate, showing a different process terminating last.	

Question 17

Consider the following Programs and Process Trees.

Program A	Program B	Program C	Program D
<pre>int main() { if (fork()) fork(); sleep(1000); }</pre>	<pre>int main() { if (!fork()) fork(); sleep(1000); }</pre>	<pre>int main() { if (fork()) if(fork()) fork(); sleep(1000); }</pre>	<pre>int main() { if (fork()) fork(); else fork(); sleep(1000); }</pre>
Tree 1	Tree 2	Tree 3	Tree 4
<pre>PID CMD 10252 -bash 10316 _ ./ft 10317 _ ./ft 10319 _ ./ft 10318 _ ./ft</pre>	<pre>PID CMD 10252 -bash 10304 _ ./ft 10305 _ ./ft 10306 _ ./ft</pre>	<pre>PID CMD 10252 -bash 10836 _ ./ft 10837 _ ./ft 10838 _ ./ft 10839 _ ./ft</pre>	<pre>PID CMD 10252 -bash 10694 _ ./ft 10695 _ ./ft 10696 _ ./ft</pre>

Each of the four Programs (A through D) produced one of the four process tree diagrams (1 through 4) when started in the background. Which Program produced which tree?

Programs	A	B	C	D
Trees				

Question 18

Consider the following program

```
const int READ_END = 0;
const int WRITE_END = 1;

static pid_t run_process(char* exe, char* arg1, int std_in, int std_out)
{
    pid_t child = fork();
    if (child == 0) { // fork child process
        char* argv[] = { exe, arg1, NULL };
        if (std_in != -1) // redirect stdin
            dup2(std_in, STDIN_FILENO);
        if (std_out != -1) // redirect stdout
            dup2(std_out, STDOUT_FILENO);
        execvp(exe, argv);
    }
    waitpid(child, NULL, 0); // wait for child
    return child;
}

int main(int ac, char* av[])
{
    int fd[2];
    pipe2(fd, O_CLOEXEC);
    // same as pipe, just closes the pipe when process calls exec

    run_process("cat", av[1], -1, fd[WRITE_END]);
    close(fd[WRITE_END]); // close parent's write end

    run_process("wc", "-m", fd[READ_END], -1);
    close(fd[READ_END]); // close parent's read end
}
```

This program is compiled with `gcc -Wall -o piping piping.c`. When run like so:

```
$ ./piping piping.c
```

```
909
```

The program outputs a result, the number of characters in the source file.

However, when run like so:

```
$ ./piping /home/aneeq/Desktop/OS/words
```

The program appears not to finish, it gets “stuck”

- (A) Explain why this program finished when run in first way but didn't finish when run the second way.

(B) Repair the program so that it completes successfully, i.e.,

```
$ ./piping /home/aneeq/Desktop/OS/words
```

```
4953680
```

Note: You should not write a program that performs the functionality of counting the number of characters in a file, rather you should make the minimum amount of changes to the given program to make it complete and perform its functionality in the intended way. The repaired program must send its data through a pipe. This can be accomplished by rearranging the system calls made by the program without adding new ones or removing any.

Question 19

Reading from file Twice:

Your friend tells you that they can open a file once but read it twice. Although you are skeptical, you decide to give it a try. Without utilizing another call to open, finish the following `double_read()` function. This function should read the specified file twice, storing the appended result as a duplicated, null-terminated string in the given buffer 'buffer'. Note: You must actually read() the file contents twice – do not just copy the data after reading the file once!

In the following, assume that all system calls succeed and that the buffer is big enough to hold the result. Read the file in a maximum of `CHUNK_SIZE` bytes at a time. While you do not necessarily need to use every line here, you are also not allowed to add semicolons to existing lines.

```
01: # define CHUNK_SIZE 1024
02:
03: void double_read(char* filename, char* buffer) {
04:     int fd = open(filename, O_RDONLY);
05:     _____;
06:     _____;
07:
08:     for (_____; _____; _____) {
09:         while(true) {
10:             _____;
11:             if (_____) break;
12:             _____;
13:         } // end of while
14:         _____;
15:     } // end of for
16:     _____;
17:     close(fd);
18: } // end of function
```

Question 20

Indicate the output produced by running the following code.

```
01: void* helper(void* args) {
02:     int *fd = (int *)args;
03:     printf("Printing from thread helper\n");
04:     dup2(*fd, STDOUT_FILENO);
05:     return NULL;
06: }
07:
08: int main() {
09:     printf("Starting main\n");
10:     int file_fd = open("test.txt", O_WRONLY | O_CREAT | O_TRUNC, 0666);
11:     int* temp_fd = (int *)malloc(sizeof(int));
12:     *temp_fd = dup(STDOUT_FILENO);
13:     pthread_t thread;
14:     dup2(file_fd, STDOUT_FILENO);
15:     pthread_create(&thread, NULL, &helper, temp_fd);
16:     pthread_join(thread, NULL);
17:     printf("Ending main\n");
18:     return 0;
19: }
```

Standard out	test.txt

Now, suppose we interchange line # 14 and 15. Now **predict the output**.

Standard out	test.txt

Question 21

Provide **3 possible outputs** of the following code. Assume all the calls i.e., `pthread_create()` and `pthread_join()` successfully execute.

```
void subtask(void* args_) {
    int idx = (int)args_;
    printf("%d ", idx);
    return;
}

int main(void) {
    pthread_t threads[6];
    for (int i = 1; i <= 6; i++)
    {
        pthread_create(&threads[i - 1], NULL, subtask, (void*)i);
        if (i % 3 == 0) {
            for (int j = i - 2; j <= i; j++)
                pthread_join(&threads[j - 1], NULL);
        }
    }
    return 0;
}
```

Output No. 1 (Any possible)	
Output No. 2 (Any possible)	
Output No. 3 (Any possible)	

Question 22

To test the functionality of multithreading you write the following code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <pthread.h>

#define WORMS 8

typedef struct {
    pthread_t tid;
    char* msg;
} pthread_args;
```

```

static void* spawn_worm(void* arg) {
    pthread_args* args = (pthread_args*)arg;
    char msg[100];
    // copies formatted string to MSG
    sprintf(msg, "Worm #%ld", args->tid);
    args->msg = msg;
}

int main() {
    pthread_args args;
    int s;
    for (int i = 0; i < WORMS; i++) {
        s = pthread_create(&args.tid, NULL, &spawn_worm, &args);
        if (s != 0) {
            return 1;
        }
    }
    for (int i = 0; i < WORMS; i++) {
        s = pthread_join(args.tid, NULL);
        if (s == 0) {
            printf("%s\n", args.msg);
        }
    }
    return 0;
}

```

You run the program assuming the following:

- Newly spawned threads start at ID 1 and increment by 1 for each additional thread.
- Your machine specification is good enough to not fail any memory allocations.

You expect that this program prints out all of the worm numbers from **1 to 8** in sequential order like:

```

OS2006@FAST ~$ gcc silkworm.c -o silkworm -lpthread
OS2006@FAST ~$ ./silkworm
| Worm 1
| Worm 2
| Worm 3
| Worm 4
| Worm 5
| Worm 6
| Worm 7
| Worm 8
OS2006@FAST ~$

```

However, upon compiling and running this program, you noticed that your output seems “**bugged**”; you get the following output:

```

OS2006@FAST ~$ gcc silkworm.c -o silkworm -lpthread
OS2006@FAST ~$ ./silkworm
|
OS2006@FAST ~$

```

Identify which lines of code (**5 lines**) should be replaced and their replacements such that the program will work as expected. You are **NOT** allowed to delete or add lines.

Old line	New Replaced Line

Question 23

Parallel File Copy:

You are making a backup copy of your entire hard drive, but you find that copying a single file at a time takes too long. You decide to put your OS knowledge to good use, and copy multiple files in parallel.

Your program takes in a non-negative integer n and two lists of strings, both of length n . The first list, **in_files**, contains a list of source files; the second list, **out_files**, contains the corresponding destination file names. Write a program to copy each source file to the corresponding destination. You must create a separate process to copy each file. The files should all be copied concurrently.

The main (parent) process must wait for all the files to be copied, and all child processes to exit, before exiting.

Fill in the blanks below to implement your parallel file copy program. You may assume calls to read and write never error. Do NOT assume they read/write the full amount.

NOTE: You should only use **AT MOST** one line per blank. Extra lines are NOT allowed. Your solution does not need to use all the blanks. You may use loops, conditional statements, and other control structures in your solution as you see fit.

```
01: #define BUFSIZE 2048
02: void copy_file(char* src, char* dst) {
03:     char buf[BUFSIZE];
04:
05:     int srcfd = open(src, O_RDONLY);
06:     int dstfd = open(dst, O_WRONLY | O_CREAT | O_TRUNC, 0644);
07:     int chunk, total;
08:
09:     while (_____) {
10:         _____;
11:         _____;
12:         _____;
13:         _____;
14:         _____;
15:         _____;
16:     } // end of while
17:
```

```

18:     close(srcfd);
19:     close(dstfd);
20: } // end of copy_file()
21:
22: void copy(int n, char** in_files, char** out_files) {
23:     pid_t children[n];
24:     int i;
25:
26:     for (i = 0; i < n; i++) {
27:         _____;
28:         _____;
29:         _____;
30:         _____;
31:         _____;
32:         _____;
33:         _____;
34:         _____;
35:         _____;
36:         _____;
37:         _____;
38:     } // end of for loop
39:
40:     _____;
41:     _____;
42:     _____;
43:     _____;
44:     _____;
45:     _____;
46: } // end of copy()
47:
48: int main() {
49:     // Example usage
50:     int n = 2;
51:     char* in_files[] = {"test1.txt", "test2.txt"};
52:     char* out_files[] = {"copy1.txt", "copy2.txt"};
53:     // test1.txt should be copied to copy1.txt
54:     // test2.txt should be copied to copy2.txt
55:     copy(n, in_files, out_files);
56:     return 0;
57: } // end of main()

```

Question 24

Santa's List

Santa is digitizing his accounting department and would like to track naughty and nice children on his computer. He has a file for each person in the form of childX.txt (e.g. child0.txt) that contains string records representing the good and bad deeds someone has done all year.

For example,

child0.txt

GOOD attended live lecture and asks questions.

GOOD finished assignments in a timely manner.

BAD didn't study for midterm.

child1.txt

BAD didn't attend any discussion.

BAD didn't help his teammates during projects.

child2.txt

GOOD answered a lot of questions on Piazza.

BAD didn't fill out course evaluations.

Fill in the code below that determines whether a child has performed more good deeds than bad deeds. As there are many children, **create a new thread for each file for concurrency**. When all threads are done counting all files, the program should write the filenames with positive karma to nice.txt and others to naughty.txt. Separate each filename by a newline with a trailing newline at the end of the file.

```
01: typedef struct thread_data {
02:     int tid;
03:     pthread_t thread;
04:     char filename[128];
05:     int karma;
06: } thread_data_t;
07:
08: void* count_good_bad(void* threadarg) {
09:     _____;
10:     _____;
11:     char* line = NULL;
12:     size_t len = 0;
13:     ssize_t read;
14:     while ((read = getline(&line, &len, fp)) != -1) {
15:         if (strncmp(line, "GOOD", 4) == 0)
16:             tdata->karma++;
17:         else if (strncmp(line, "BAD", 3) == 0)
18:             tdata->karma--;
19:     } // end of while
20:     pthread_exit(NULL);
21: }
22:
23: int main(int argc, char* argv[]) {
24:     int num_children = atoi(argv[1]);
25:     thread_data_t thread_data_array[num_children];
```

```

26:     for (int i=0; i<num_children; i++) {
27:         thread_data_array[i].tid = I;
28:         sprintf(thread_data_array[i].filename, "child%d.txt", i);
29:         thread_data_array[i].karma = 0;
30:         _____;
31:     }
32:
33:     for (int i=0; i<num_children; i++)
34:         pthread_join(_____, NULL);
35:
36:     FILE* fp_nice = fopen("nice.txt", "w");
37:     FILE* fp_naughty = fopen("naughty.txt", "w");
38:
39:     for (_____; _____; _____) {
40:         if (_____)
41:             _____;
42:         else
43:             _____;
44:     }
45:     fclose(fp_nice);
46:     fclose(fp_naughty);
47:     return 0;
48: }

```

With the above example, the files would look like:

nice.txt

child0.txt
child2.txt

naughty.txt

child1.txt

Also, answer the following Questions:

- (A) What would happen if we did not call `pthread_join()` ?
- (B) How would you implement this using processes rather than threads ?

Question 25

Spell Finder

Harry has a big test coming up at LogPorts Castle, and he hasn't read the textbook yet! He needs to be able to perform a specific spell, but he doesn't know where it is in the textbook.

Harry has asked you, a computer magic major at LogPorts, to write a multiprocessing program for him that can rapidly find the spell in the textbook. Your solution must create `num_processes` processes, and each process should search an equal portion of the textbook using the `search_for_spell()` function.

You can assume that `num_processes` is a power of 2, and that `textbook_size` is a multiple of `num_processes`.

NOTE: For each blank, you can use as many lines as you need to solve the problem. You may be able to leave some blanks empty. You can assume that all syscalls will not error. **Your solution must run all processes in parallel, A solution in which processes are running serially is NOT allowed.**

```
/* This function will read the textbook file into array `arr`. You can assume that the
textbook will be exactly `textbook_size` characters long. Assume that this is already
implemented. */
```

```
void read_textbook_into_array(char *arr, int textbook_size);
```

```
/* This function will search for Harry's spell in array `arr`, and print the spell if
found. The function will only search for the spell in the subarray between `start_index`
(inclusive) and `end_index` (exclusive). Assume that this is already implemented. */
```

```
search_for_spell(char *arr, int start_index, int end_index);
```

```
01: void process_array (int num_processes, int textbook_size) {
02:     char* arr = malloc(sizeof(char) * textbook_size);
03:     read_textbook_into_array(arr, textbook_size);
04:
05:     int start_index = 0;
06:     int end_index = textbook_size;
07:
08:     for (_____; _____; _____) {
09:         pid_t pid = fork();
10:         if (pid == 0) {
11:             _____;
12:             _____;
13:             _____;
14:             _____;
15:         }
16:         else {
17:             _____;
18:             _____;
19:         }
20:         search_for_spell(arr, start_index, end_index);
21:     }
```

Also, answer the following Questions:

- (A) Why this program is not memory-efficient with respect to the size of textbook?
- (B) Neville, a computer magic minor, recommends that you use multithreading instead. Justify his recommendation.

Question 26

In the below program, several threads are created. Which of the following statements (**select all possible**) in foo always print the same memory address when evaluated by different threads in same process?

```
01: int global;
02:
03: void* foo(void* arg) {
04:
05:     printf("%p\n", &foo);
06:     printf("%p\n", &global);
07:     printf("%p\n", &arg);
08:     printf("%p\n", arg);
09:     return NULL;
10:
11: }
12:
13: int main() {
14:     void* hmem = malloc(1);
15:     for (int i=0; i<3; i++) {
16:         pthread_t pid;
17:         pthread_create(&pid, NULL, foo, hmem);
18:     }
19:     return 0;
20: }
```

☐	printf("%p\n", &foo);	☐	printf("%p\n", &global);
☐	printf("%p\n", arg);	☐	printf("%p\n", &arg);
☐	None of the above		

Question 27

Beaver Bother Logs

You are writing a program for the beavers of Beaver Bother that will automatically sort log entries into separate log files based on a simple protocol: the first character of each log entry will be a digit representing the index of the destination log file. For example, the entry **2 - Beaver Bother will be saved to log2.txt**, while the entry **0 - Bothering Beavers will be saved to log0.txt**. Log entries will be collected from standard input. You've written a function, `get_entry`, to get one log entry from **stdin** and **save it to a null-terminated buffer**.

IMPORTANT: Even though each entry is null-terminated, the null terminator should not be written to the log file.

You also decide that the main process of the program should spawn a child process for each log file (to handle file IO), and you plan to use pipes to forward log entries to each child process. Fill in the missing code below to finish your program!

NOTE: You should only use one line per blank. Extra lines are NOT allowed.

```
01: #define BUFFER_LEN 1024
02:
03: int get_entry(FILE * infile, char* buf, size_t buflen) {
04:     size_t index = 0;
05:     char ch = fgetc(infile);
06:     int log_index = ch - '0';
07:     buf[0] = '\\0';
08:     while ((index < buflen - 2) && (ch != '\\n')) {
09:         buf[index++] = ch;
10:         buf[index] = '\\0';
11:         ch = fgetc(infile);
12:     }
13:     buf[index++] = '\\n';
14:     buf[index] = '\\0';
15:     return log_index;
16: }
17:
18: typedef struct pipe_fds {
19:     int fds[2];
20: } pipe_fds;
21:
22: int main(int argc, char** argv) {
23:     int num_logs = atoi(argv[1]);
24:     pipe_fds log_fds[num_logs];
25:     pid_t pid;
26:     int log_index;
27:     for (log_index = 0; log_index < num_logs; log_index++) {
28:         _____;
29:         if ((pid = fork()) == 0)
30:             _____;
31:     }
32:     if (pid != 0) {
33:         char str[BUFFER_LEN];
34:         while (1) {
35:             log_index = get_entry(stdin, str, BUFFER_LEN);
36:             size_t bytes_written;
37:             size_t total = 0;
38:             size_t str_length = strlen(str);
39:             int wfd = log_fds[log_index].fds[1];
40:             while ((bytes_written = write(_____))) {
41:                 _____;
42:             }
43:         }
44:     } // end of if
45:     else {
46:         char filename[BUFFER_LEN];
47:         sprintf(filename, "log%d.txt", log_index);
48:         int file_fd = open(filename, O_RDWR | O_CREAT, 0666);
49:         char read_buf[BUFFER_LEN];
```

```

50:     size_t bytes_read;
51:     int rfd = _____;
52:
53:     while (bytes_read = read(rfd, read_buf, BUFFER_LEN)) {
54:         size_t bytes_written;
55:         size_t total = 0;
56:         while (bytes_written = write(_____)) {
57:             _____;
58:         }
59:     }
60: } // end of else
61: return 0;
62: } // end of main

```

Question 28

In the following code snippet, **Assume at least two individual calls to fork() succeed.**

```

#include <sys/types.h>
#include <unistd.h>

int main() {
    for (int i = 0; i < 3; i++) {
        fork();
    }
    return 0;
}

```

How many new processes could be created?		How many times could fork() be called?	
☐	0	☐	1
☐	2	☐	3
☐	4	☐	5
☐	6	☐	7
☐	8	☐	8

Note: Select all possible options.

Question 29

In the following code snippets assume all calls to `read( )`, `write( )` and `fork( )` succeed. Suppose that `montymole.txt` was empty before running these blocks of code. All codes are separate from each other.

You have to select all possible options that can be content of `montymole.txt` after each code block execution.

<pre>int main(int argc, char** argv) { int fd1 = open("montymole.txt", O_WRONLY); int fd2 = open("montymole.txt", O_WRONLY); write(fd1, "mole", 4); write(fd2, "whack", 5); write(fd2, "mole", 4); write(fd1, "mole", 4); write(fd1, "mole", 4); close(fd1); close(fd2); }</pre>	<table><tr><td>☐</td><td>whacmolemole</td><td>☐</td><td>whacmolek</td></tr><tr><td>☐</td><td>whackmolemole</td><td>☐</td><td>Nothing</td></tr><tr><td>☐</td><td colspan="3">molewhackmolemole</td></tr></table>	☐	whacmolemole	☐	whacmolek	☐	whackmolemole	☐	Nothing	☐	molewhackmolemole		
☐	whacmolemole	☐	whacmolek										
☐	whackmolemole	☐	Nothing										
☐	molewhackmolemole												
<pre>void new_thread(void* arg) { int* fd = (int*)arg; write(*fd, "whackwhackmole", 14); } int main(int argc, char** argv) { int fd1 = open("montymole.txt", O_WRONLY); pthread monty_mole; pthread_create(&monty_mole, NULL, new_thread, (void*)&fd1); write(fd1, "mole", 4); close(fd1); }</pre>	<table><tr><td>☐</td><td>molewhack</td><td>☐</td><td>whackwhackmole</td></tr><tr><td>☐</td><td>whackwhackmolemole</td><td>☐</td><td>molewhackwhackmole</td></tr><tr><td>☐</td><td colspan="3">molewhackwhackwhack</td></tr></table>	☐	molewhack	☐	whackwhackmole	☐	whackwhackmolemole	☐	molewhackwhackmole	☐	molewhackwhackwhack		
☐	molewhack	☐	whackwhackmole										
☐	whackwhackmolemole	☐	molewhackwhackmole										
☐	molewhackwhackwhack												
<pre>void fork_p(pid_t pid, int fd1) { if (pid == 0) { write(fd1, "whack", 5); } else { write(fd1, "mole", 4); } } int main(int argc, char** argv) { int fd1 = open("montymole.txt", O_WRONLY); fork_p(fork(), fd1); write(fd1, "mole", 4); close(fd1); }</pre>	<table><tr><td>☐</td><td>molekmole</td><td>☐</td><td>whacmmole</td></tr><tr><td>☐</td><td>whackmole</td><td>☐</td><td>molemolee</td></tr><tr><td>☐</td><td colspan="3">whackmolemole</td></tr></table>	☐	molekmole	☐	whacmmole	☐	whackmole	☐	molemolee	☐	whackmolemole		
☐	molekmole	☐	whacmmole										
☐	whackmole	☐	molemolee										
☐	whackmolemole												

Question 30

Consider the following piece of code, which can be executed by one or more threads:

```
x = x + 1;  
y = x + y;
```

Assumptions:

- Before any thread starts running, both x and y are initialized to 1. After that x and y are only updated by the threads executing the above code segment.
- A thread can be preempted at any point during its execution.
- Each of the two instructions is atomic.

(A). Assume two copies of the above piece of code run concurrently in two threads. Write down all Possible values after both threads finish, one per column
(Number of outputs might be smaller than the number of columns.)

x								
y								

(B). Same question as above, but now assume that three copies of the code segment in three threads.

x								
y								

(C). Assume 10 copies of the same program run concurrently. What is the maximum value of y?

(D). Assume 10 copies of the same program run concurrently. What is the minimum value of y?

Question 31

Consider the following code, assume all fork() and other calls succeed.

```
01: void* rem(void *args) {  
02:   printf("Blue: %d\n", *((int*) args));  
03:   exit(0);  
04: }  
05: void* ram(void *args) {  
06:   printf("Pink: %d\n", ((int*) args)[0]);  
07:   return NULL;  
08: }
```



```

09: int main(void) {
10:     pid_t pid; pthread_t pthread; int status; //declaring vars
11:
12:     int fd = open("emilia.txt", O_CREAT|O_TRUNC|O_WRONLY, 0666);
13:     int *subaru = (int*) calloc(1, sizeof(int));
14:     printf("Original: %d\n", *subaru);
15:
16:     if(pid = fork()) {
17:         *subaru = 1337;
18:         pid = fork();
19:     }
20:     if(!pid) {
21:         pthread_create(&pthread, NULL, ram, (void*) subaru);
22:     } else {
23:         for(int i = 0; i < 2; i++)
24:             waitpid(-1, &status, 0);
25:         pthread_create(&pthread, NULL, rem, (void*) subaru);
26:     }
27:     pthread_join(pthread, NULL);
28:     if(*subaru == 1337)
29:         dup2(fd, fileno(stdout));
30:     printf("All done!\n");
31:     return 0;
32: }

```

- (A). Including the original process, how many processes are created?
- (B). Including the original thread, how many threads are created?
- (C). Provide all possible outputs in standard output. If there are multiple possibilities, write all of them.
- (D). Provide all possible contents of emilia.txt. If there are multiple possibilities, write all of them.
- (E). Suppose we deleted line 28, how would the contents of emilia.txt change (if they do)?
- (F). What if, in addition to doing the change in part (E), we also move line 12 between lines 19 and 20? What would emilia.txt look like then?

Question 32

Consider the following C program.

```

int  thread_count = 0;
void* thread_start(void* arg) {
    thread_count++;
    if (thread_count == 3) {
        char* argv[] = { "/bin/ls", NULL };
        execv(*argv, argv);
    }
}

```

```

    printf("Thread:  %d\n", thread_count);
    return  NULL;
}

int  main(int  argc, char* argv[]) {
    int  i;
    for (i = 0; i < 10; i++) {
        pthread_t* thread = malloc(sizeof(pthread_t));
        pthread_create(thread, NULL, &thread_start, NULL);
        pthread_join(*thread, NULL);
    }
    return  0;
}

```

- (A). When you run the program, what will be the output.
- (B). If we removed the line `pthread_join(*thread, NULL)`, could the output change? If so, explicitly describe the way(s) it might differ.

Question 33

Consider the following two C programs

```

int i = 100;
char* buf;

void process() {
    pid_t pid;
    int status, rd;
    int j = 1;

    buf = strcpy(malloc(100), " boring ");
    pid = fork();

    if (pid != 0) {
        printf("P Parent : j = %d \n", j);
        i = 162;
        j = 2;

        printf("P Parent : j = %d \n", j);
        printf("P Parent : %s\n ", buf);
        j = 4;
        wait(&status);
    }
    else {
        printf("P Child : i = %d, j = %d\n", i, j);
        printf("P Child : i = %d, j = %d\n", i, j);

        j = 3;
        strcpy(buf, " cool ");
        exit(0);
    }
}

```

```

int i = 100;
char* buf;

void* tfun(void* noarg) {
    int j = 0;
    printf("T Child : i = %d, j = %d\n", i, j);
    printf("T Child : i = %d, j = %d\n", i, j);

    j = 3;
    strcpy(buf, " cool ");
    pthread_exit(NULL);
}

void thread() {
    pthread_t tid;
    int j = 1;
    buf = strcpy(malloc(100), " boring ");

    pthread_create(&tid, NULL, tfun, NULL);
    printf("T Parent : j = %d \n", j);
    i = 162;
    j = 2;

    printf("T Parent : j = %d \n", j);
    printf("T Parent : %s\n ", buf);
    j = 4;
    pthread_join(tid, NULL);
}

```

The above two programs are nearly identical process based and thread based programs. For each question, answer YES or NO and provide a short, crisp explanation. Assume all functions will succeed.

- (A). Can the output of the process-based program ever include (meaning they appear in this order, possibly with other output interleaved) the following? Why?

```
P Parent: j = 1
P Child: i = 100, j = 1
P Parent: j = 2
```

- (B). Can the output of the thread-based program ever include the following? Why?

```
T Parent: j = 1
T Child: i = 100, j = 1
T Parent: j = 2
```

- (C). Can the output of the process-based program ever include the following? Why?

```
P Parent: j = 1
P Child: j = 2
P Child: i = 100, j = 1
```

- (D). Can the output of the thread-based program ever include the following? Why?

```
T Parent: j = 1
T Parent: j = 2
T Parent: i = 100, j = 1
```

- (E). Can the output of the process-based program ever include the following? Why?

```
P Parent: cool
```

- (F). Can the output of the process-based program ever include the following? Why?

```
T Parent: cool
```

Question 34

Consider the following C code

```
#define NUM_THREADS 3

void* thread_function(void* arg) {
    int* arg = (int*)arg;
    printf("Thread %d: Running\n", *arg);

    if (!fork()) {
        printf("Where am I, who called me here /: did you %d\n", getppid());
    }
}
```

```

        printf("Well then will you reap me OR I should make more threads :)\n");
        if (*arg < 1)
            exit(0);
    }
    else
    {
        sleep(2); // until child completes its execution
        printf("What is my ID /:) is It %d\n", getpid());
    }

    pthread_exit(NULL);
}

int main() {
    pthread_t threads[NUM_THREADS];
    int thread_args[NUM_THREADS];
    int i, status;

    for (i = 0; i < NUM_THREADS; i++) {
        thread_args[i] = i + 1;
        status = pthread_create(&threads[i], NULL, thread_function,
                                (void*)&thread_args[i]);

        wait(NULL);        // can I reap that child of yours
    }

    pthread_exit(NULL);

    printf("Byee\n");

    return 0;
}

```

No. of processes created?	
No. of threads created?	
No. of zombie processes just before the program exits (Main program)?	
Will printf("Byee\n") execute?	
Suppose if we remove sleep() then will the printf() statements always execute in child?	
No. of zombie processes if wait() is removed from for loop and written in place of sleep()?	
Is there a possibility that this program terminates without showing any output, given that all syscalls i.e., pthread_create, etc succeed?	
Write pid of all processes that have one or more children. Given that starting (main) process has a pid of 8342 and any child of a process has (pidOfparent + 1) as it's pid, which surely increments as no of childs increase. No two processes have same pid, in case of clash give next pid available.	

Question 35

Consider the following C code.

```
#define NUM_THREADS 3

void* thread_function(void* arg) {
    int* arg = (int*)arg;
    printf("Thread %d: Running\n", *arg);

    sleep(10);

    printf("Here we are from process %d\n", getpid());

    pthread_exit(NULL);
}

int main() {
    pthread_t threads[NUM_THREADS];
    int thread_args[NUM_THREADS];
    int i, status;

    for (i = 0; i < NUM_THREADS; i++) {
        thread_args[i] = i + 1;
        status = pthread_create(&threads[i], NULL, thread_function,
                               (void*)&thread_args[i]);
    }

    // suppose this fork call executes before
    // any thread has completed its sleep(10)
    if (!fork())
    {
        printf("Do I have your threads, hahaha\n");
        printf("Can I join your threads or MINE??\n");
        for (i = 0; i < NUM_THREADS; i++) {
            if (pthread_join(&threads[i], NULL))
                printf("No i Can't join..%d", i);
            else
                printf("Yes I can join..%d", i);
        }
        pthread_exit(NULL); // what will this do? hmmm...
    }
    else
    {
        wait(NULL);
        for (i = 0; i < NUM_THREADS; i++) {
            if (pthread_join(&threads[i], NULL))
                printf("No i myself Can't join..%d", i);
            else
                printf("Yes I mself can join..%d", i);
        }
    }
}
```

```

}

printf("Bye\n");
pthread_exit(NULL);

return 0;
}

```

Answer the following Questions

- A) Write any three (3) possible outputs of the code, provided parent process has 8342 and child has 8343 as their pid respectively. **Only consider printf statements in the function.**

--	--	--

- B) Give answers of following

Total No. of threads created?	
How many time printf("Bye\n") will be executed?	
How many calls to pthread_join can produce error?	
Suppose before fork call thread with threads[0] has completed it's execution. Now How many threads can be at max?	
Suppose there is a fork call just before for loop written as fork(); How many possible (Max) threads can be created?	
Considering the above scenario, can there be any case that program terminates and all threads have not completed their execution?	
Suppose in the original scenario, if all pthread_join calls fail, both in parent and child, can there be a possibility that program show only Bye as output?	

Question 36

Scheduling

All timeslice-based algorithms have a timeslice of one unit. The currently running thread is not in the ready Queue while it is running. An arriving thread is run at the beginning of its arrival time, if the scheduling policy allows it.

Fill in ALL blanks in EACH table – each blank has an unambiguous answer.

For the missing schedulers, the possibilities are **SRTF, RR, and Priority**. *Priority is a preemptive scheduler.*

Hint: Fill in the entry time (below) for Thread C first!

Entry Times	
A	1
B	2
C	
D	8

Priorities	
A	3
B	4
C	5
D	6

↓ Current Time Scheduler →	Currently Scheduled Process		
	FIFO		
1	A	A	A
2	A	A	B
3	A	A	B
4	B		
5	B		
6	B		
7	C		
8	D		
9	D		
10	D		
Avg. Turnaround Time	3.5		

Question 37

Consider the following table of processes and their associated arrival and running times.

Process ID	Arrival Time	CPU Running Time
Process A	0	2
Process B	1	6
Process C	4	1
Process D	7	4
Process E	8	3

Show the scheduling order for these processes under 3 policies: First Come First Serve (**FCFS**), Shortest-Remaining-Time-First (**SRTF**), Round-Robin (**RR**) with time slice quantum = 1.

Assume that context switch overhead is 0, that new processes are available for scheduling as soon as they arrive, and that new processes are added to the **head** of the queue **except for FCFS, where they are added to the tail**.

Time Slot	FCFS	SRTF	RR
0			
1			
2			
3			
4			
5			
6			
7			
8			
9			
10			
11			
12			
13			
14			
15			

Question 38

Suppose we have the following set of processes, with their respective CPU burst times and priorities (here, processes with a higher priority value have higher priority).

Process ID	Arrival Time	CPU Burst (running) Time	Priority
A	0	3	4
B	1	1	2
C	3	5	5
D	6	2	6
E	8	2	1

For each of the three scheduling algorithms, fill in the the table below with the processes that are running on the CPU at each time tick. Assume the following:

- The time quantum for RR is 1 clock tick and when RR quantum expires, the currently running thread is added to the end of the ready list before any newly arriving threads.
- Assume that context switch overhead is 0, and that new processes are available for scheduling as soon as they arrive.

Time	RR	SRTF	Priority
0			
1			
2			
3			
4			
5			
6			
7			
8			
9			
10			
11			
12			
Average Wait Time			

Question 39

CPU scheduling.

Consider the following **single-threaded** processes, arrival times, and CPU processing requirements:

Process ID (PID)	Arrival Time	Processing Time
1	0	6
2	2	4
3	3	5
4	6	2

A) For each scheduling algorithm, fill in the table with the ID of the process that is running on the CPU. Each row corresponds to a time unit.

- For time slice-based algorithms, assume one unit time slice.
- When a process arrives it is immediately eligible for scheduling, e.g., process 2 that arrives at time 2 can be scheduled during time unit 2.
- If a process is preempted, it is added at the **tail of the ready queue**.

Time	FIFO	RR	SJF
0			
1			
2			
3			
4			
5			
6			
7			
8			
9			
10			
11			
12			
13			
14			
15			
16			

B) Calculate the response times of individual processes for each of the scheduling algorithms. The response time is defined as the time a process takes to complete after it arrives.

	PID 1	PID 2	PID 3	PID 4
FIFO				
RR				
SJF				

C) Consider same processes and arrival times, but assume now a processor with **two** CPUs. Assume CPU 0 is busy for the first two time units. For each scheduling algorithm, fill in the table with the ID of the process that is running on each CPU.

- For any non-time slice-based algorithm, assume that once a process starts running on a CPU, it keeps running on the same CPU till the end.
- If both CPUs are free, assume CPU 0 is allocated first.

Time	CPU #	FIFO	RR	SJF
0	0			
	1			
1	0			
	1			
2	0			
	1			
3	0			
	1			
4	0			
	1			
5	0			
	1			
6	0			
	1			
7	0			
	1			
8	0			
	1			
9	0			
	1			
10	0			
	1			

Question 40

Advanced Bike-share Scheduling

In this problem, we want you to apply what you have learned about scheduling algorithms to scheduling resources in a new domain – bike-share scheduling. Ford GoBikes are now in Islamabad and they need **your** help in figuring out how to schedule the resources in their system.

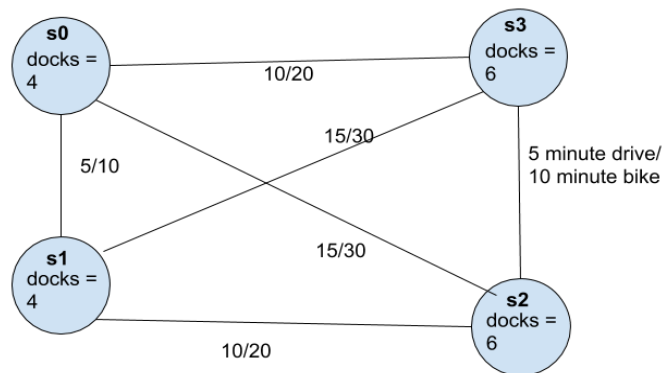
In a bike-share system, the **bikes** are parked in **stations** each with some number of **docks** for bikes. A potential **rider** walks up to a station, checks out a bike from a dock, rides to a destination station, returns the bike to an empty dock, and walks off. In order to avoid blocking riders, the bike-share operator periodically rebalances bikes by driving around in a **van** and adding or removing bikes from stations depending on demand.

Important Rebalancer Assumptions:

- Van has infinite capacity and infinite bikes and starts at s0 at 7:05
- Once the van arrives at a station, it stays there for 5 minutes to perform the rebalancing
- Multi-level priority uses the demand - (context switch time/10) to determine dynamic priority
- Pre-emptive algorithms pick up or drop off one bike on every visit; otherwise full rebalance to optimal state on every visit
- the optimal state is 50% full, every station starts in optimal state
- the demand can be represented by $\text{abs}(n_bikes - n_empty_slots)/docks$, and
- the context switch time is the drive time between stations + unloading/reloading time

Trips by riders

Start station	Start time	End station	End time
s1	7:00	s2	7:20
s3	7:05	s0	7:25
s3	7:10	s3	7:30
s3	7:20	s2	7:30



- A) Fill out the table in the given on next page with the location of the van in each time slot, assuming each of the classic scheduling algorithms applied to this new context. Use **T** if the van is in transit. **We have filled in Round-Robin for you.**

Time	FCFS	Round-robin	Most imbalanced first (non- preempt)	Most imbalanced first (preempt)	Multi-level priority
7:06		T			
7:11		s1			
7:16		T			
7:21		T			
7:26		T			
7:31		s3			
7:36		T			
7:41		s2			
7:46		T			
7:51		T			
7:56		T			
8:01		s0			
8:06		T			
8:11		T			
8:16		s3			
8:21		T			
8:26		s2			
8:31					

B) Briefly, in two to three sentences, explain what is common among the algorithms that work well in this scenario.

C) Briefly, in two to three sentences, explain; what are the characteristics of the problem that makes these algorithms particularly suitable.